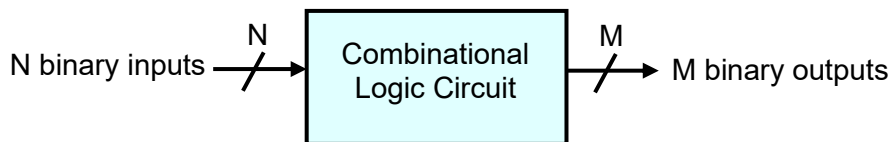


Digital Electronics
Part 3 – Combinational Circuits
Stéphane Le Goff

2026

3. Combinational Logic Circuits

Combinational logic circuits are logic circuits whose outputs at any time depend only on the inputs at the same time.



• Describing Combinational Logic Circuits Algebraically

Any combinational logic circuit, no matter how complex, can be designed using adequate logic gates. The start of the digital design process is generally the truth table.

As an illustration, let us consider the design of a majority circuit with three inputs. Its input is 1 if there are more inputs at 1 than at 0, and its output is 0 otherwise. Its truth table is therefore given in the table below.

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

From the truth table, we read that

$$\begin{aligned}
 X = 1 \text{ if } & (A = 0) \text{ AND } (B = 1) \text{ AND } (C = 1) \\
 & \text{OR } (A = 1) \text{ AND } (B = 0) \text{ AND } (C = 1) \\
 & \text{OR } (A = 1) \text{ AND } (B = 1) \text{ AND } (C = 0)
 \end{aligned}$$

$$\text{OR } (A = 1) \text{ AND } (B = 1) \text{ AND } (C = 1).$$

This is equivalent to

$$\begin{aligned} X = 1 \text{ if } & (\bar{A} = 1) \text{ AND } (B = 1) \text{ AND } (C = 1) \\ \text{OR } & (A = 1) \text{ AND } (\bar{B} = 1) \text{ AND } (C = 1) \\ \text{OR } & (A = 1) \text{ AND } (B = 1) \text{ AND } (\bar{C} = 1) \\ \text{OR } & (A = 1) \text{ AND } (B = 1) \text{ AND } (C = 1). \end{aligned}$$

Therefore, we can write $X = 1$ if $(\bar{A}BC = 1)$ OR $(A\bar{B}C = 1)$ OR $(AB\bar{C} = 1)$ OR $(ABC = 1)$, which is equivalent to the following Boolean expression:

$$X = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC.$$

This equation can thus be implemented by using three inverters, four 3-input AND gates, and one 4-input OR gate.

This will work fine. However, we should try to simplify this equation as much as possible. If we can do so, this will certainly lead to a reduction in the complexity of the circuit required to implement the equation.

• Boolean Theorems

Investigating the various Boolean theorems can help us simplify logic expressions and logic circuits. To do so, one can make use of several Boolean algebra theorems that are listed below. Feel free to check that all of them are indeed valid by considering examples.

- (1) $X \cdot 0 = 0,$
- (2) $X \cdot 1 = X,$
- (3) $X \cdot X = X,$
- (4) $X \cdot \bar{X} = 0,$
- (5) $X + 0 = X,$
- (6) $X + 1 = 1,$
- (7) $X + X = X,$

- (8) $X + \overline{X} = 1$,
- (9) $X + Y = Y + X$ (commutative law for the OR function),
- (10) $X \cdot Y = Y \cdot X$ (commutative law for the AND function),
- (11) $X + (Y + Z) = (X + Y) + Z = X + Y + Z$ (associative law for the OR function),
- (12) $X \cdot (Y \cdot Z) = (X \cdot Y) \cdot Z = X \cdot Y \cdot Z$ (associative law for the AND function),
- (13) $X \cdot (Y + Z) = X \cdot Y + X \cdot Z$,

De Morgan's Theorems: De Morgan's theorems are very useful in simplifying expressions in which a product or sum of variables is inverted. These theorems provide us with two powerful tools of Boolean algebra:

$$(14) \quad \overline{X + Y} = \overline{X} \cdot \overline{Y},$$

$$(15) \quad \overline{X \cdot Y} = \overline{X} + \overline{Y}.$$

Theorem (14) says that when the OR of two variables is inverted, this is the same as inverting each variable individually and then ANDing these inverted variables.

Theorem (15) says that when the AND of two variables is inverted, this is the same as inverting each variable individually and then ORing them.

Example: $X = \overline{(\overline{A} + C) \cdot (B + \overline{D})} = \overline{\overline{A} + C} + \overline{B + \overline{D}}$ (using Theorem 15) $= A \cdot \overline{C} + \overline{B} \cdot D$ (using Theorem 14).

De Morgan's theorems are also applicable with more than two variables.

Another rule that can prove very useful in practice is

$$(16) \quad X + \overline{X} \cdot Y = X + Y.$$

To demonstrate it, we can use the truth table shown below and check that the last two columns are indeed identical.

X	Y	$\bar{X}$	$\bar{X} \cdot Y$	$X + \bar{X} \cdot Y$	$X + Y$
0	0	1	0	0	0
0	1	1	1	1	1
1	0	0	0	1	1
1	1	0	0	1	1

In practice, applying these sixteen rules to simplify a particular Boolean expression may not always be straightforward. To illustrate that statement, let us consider the equation that we previously obtained for our majority circuit example and attempt to simplify it by applying any of the sixteen Boolean rules/theorems:

$$X = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC.$$

There are many ways to do it, and many of them will not necessarily lead to the simplest possible expression. As yours truly already knows the answer, he suggests to proceed to follow:

$$X = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC + ABC + ABC \text{ (rule 7: } X + X = X\text{)},$$

which then leads to

$$X = (A + \bar{A})BC + (C + \bar{C})AB + (B + \bar{B})AC \text{ (rule 13: } X \cdot (Y + Z) = X \cdot Y + X \cdot Z\text{)}.$$

Finally, we obtain

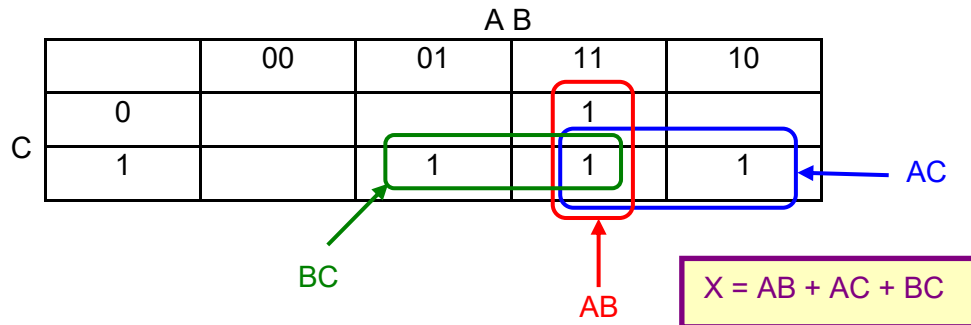
$$X = BC + AB + AC \text{ (rule 8: } X + \bar{X} = 1 \text{ and rule 2: } X \cdot 1 = X\text{)}.$$

This rather simple result strongly suggests that the simplest possible Boolean expression has been obtained. But, in general, it is not always easy to tell whether the final result could be further simplified or not.

Therefore, most designers would rather use a Karnaugh map to do the job. A Karnaugh map provides a pictorial method of grouping together expressions with common factors and therefore eliminating unwanted variables.

The Karnaugh map can also be described as a special arrangement of a truth table.

As an example, the Karnaugh map corresponding to the majority circuit previously considered is as follows:



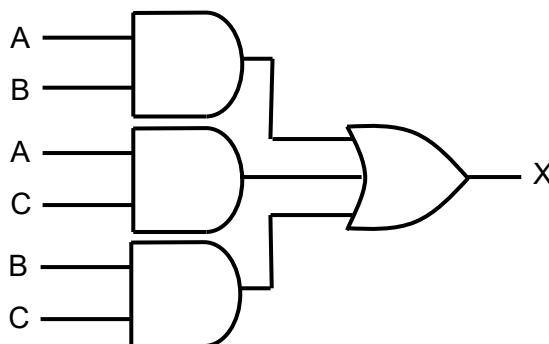
The core principle behind a Karnaugh map is that two neighbouring cells in it only differ by one bit. Therefore, if the circuit output equals one for two neighbouring cells, then the bit that differs can be discarded from that term as it is simply irrelevant. That comes from the fact that $X \cdot Y + X \cdot \bar{Y} = X \cdot (Y + \bar{Y}) = X \cdot 1 = X$.

In a Karnaugh map, the game thus consists of creating groups of 2, 4, 8, etc. neighbouring ones and associating each group with the inputs that are common in that particular group while discarding the inputs that differ in the latter. As there are many possibilities, the best way to be good at this “game” is to practice using as many examples as possible.

Let us now come back to our initial majority circuit example. The Karnaugh map technique has clearly shown us what the simplest possible Boolean expression is:

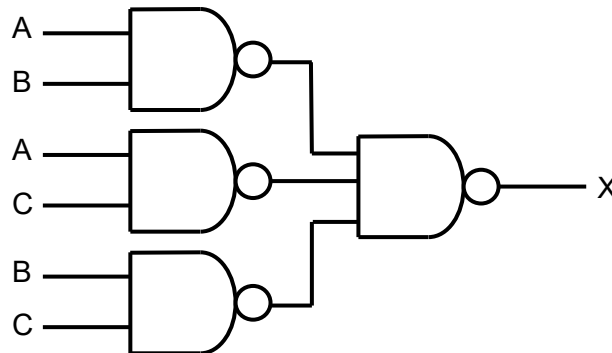
$$X = AB + AC + BC.$$

The corresponding logic circuit for our three-input majority circuit is depicted below.



S

Note that, since De Morgan's theorems tell us that $X = AB + AC + BC = \overline{\overline{AB} \cdot \overline{AC} \cdot \overline{BC}}$, we can also implement the majority circuit by using only NAND gates, as shown below.

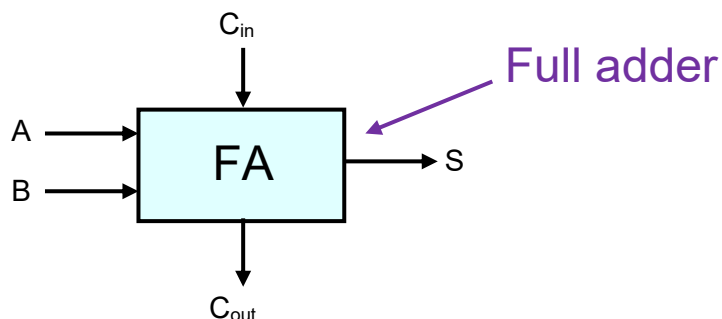


It is important to be aware that an AND-OR structure can actually be replaced by a NAND-NAND arrangement because NAND gates are significantly faster and simpler to implement than both AND and OR logic gates in CMOS technology (complementary MOSFET, the dominant technology for designing digital circuits).

Let us now consider several detailed examples of combinational logic circuits in order to illustrate the whole design process.

Example 1: Design of a full adder.

A full adder (FA) is a circuit that computes the sum of three bits, generally denoted as A, B, and C_{in} for reasons that will become obvious later on. A full adder must thus have two outputs, namely the sum S and a carry C_{out} .



The full adder is a fundamental circuit in digital electronics since it is often used as a building block for designing a multitude of arithmetic circuits such as adders, subtractors, and multipliers.

The truth table of a full adder is shown below.

A	B	C _{in}	S	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Therefore, the Karnaugh map for output S is as follows:

	A B			
	00	01	11	10
C _{in} 0		1		1
C _{in} 1	1		1	

$$S = \overline{A}\overline{B}C_{in} + \overline{A}B\overline{C_{in}} + AB\overline{C_{in}} + A\overline{B}C_{in} = \overline{C_{in}}(\overline{A}B + A\overline{B}) + C_{in}(AB + \overline{A}\overline{B})$$

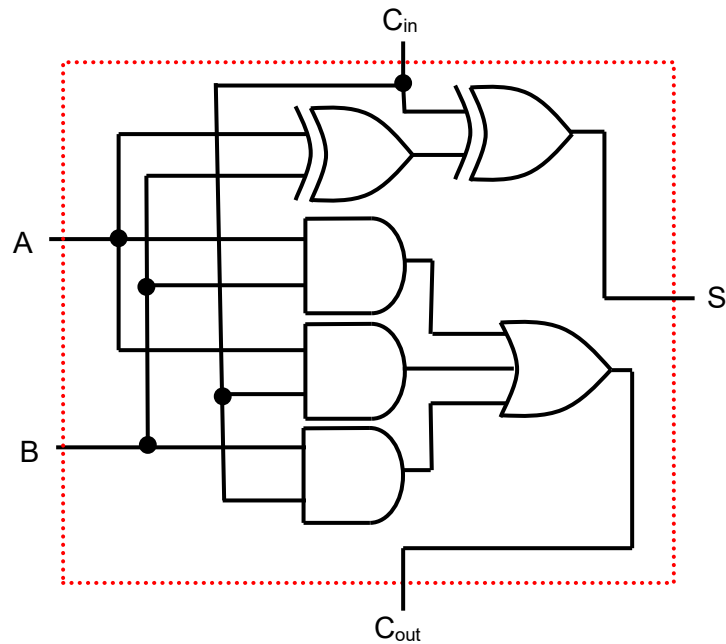
$$S = \overline{C_{in}}(A \oplus B) + C_{in}(\overline{A \oplus B}) = C_{in} \oplus (A \oplus B)$$

The Karnaugh map for output C_{out} is as follows:

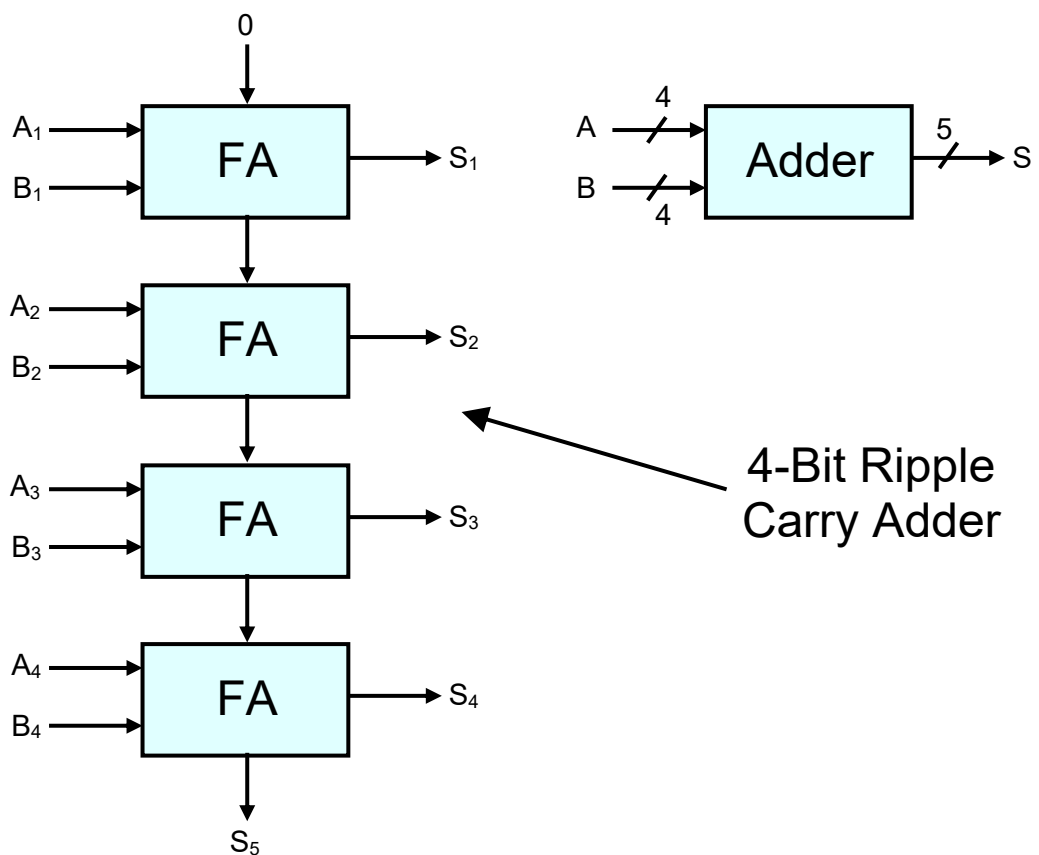
	A B			
	00	01	11	10
C _{in} 0			1	
C _{in} 1		1	1	1

$$C_{out} = AB + AC_{in} + BC_{in}$$

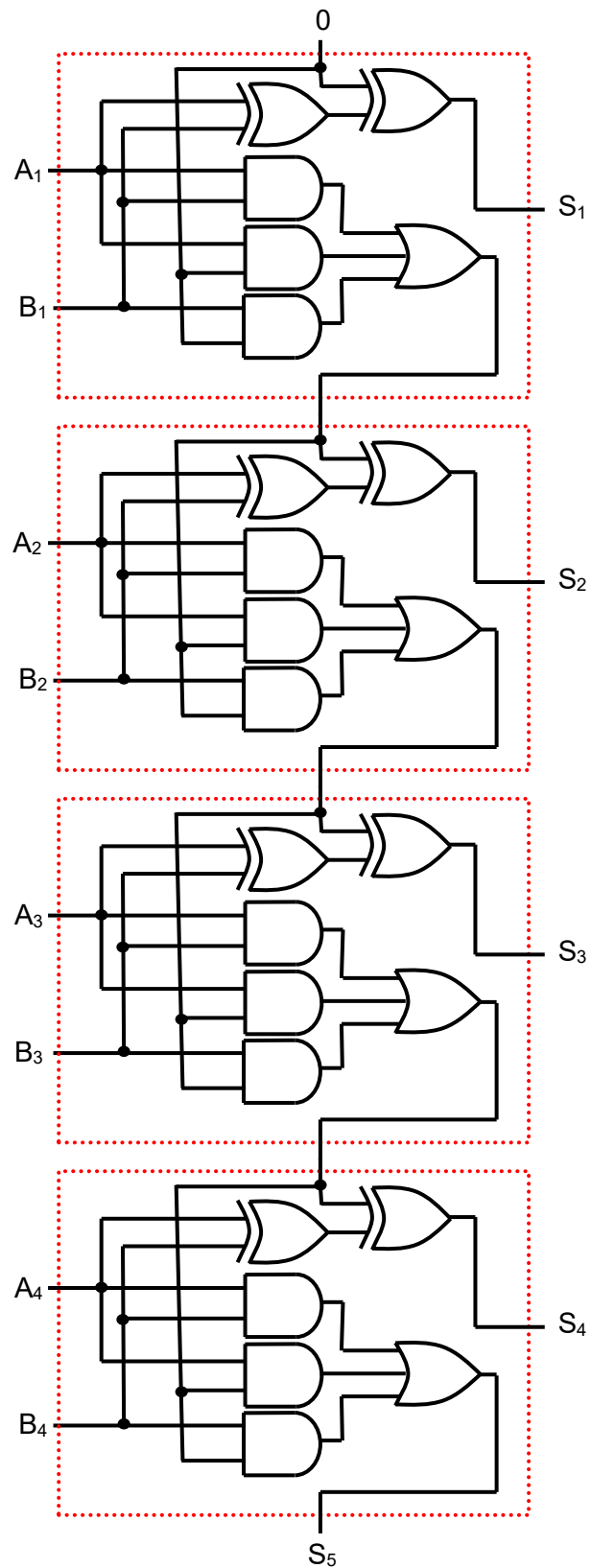
The logic diagram of a full adder is shown below.



The full adder can be used as a basic building block for designing an n -bit adder, i.e. a combinational logic circuit that performs the addition of two n -bit numbers. We provide below an example for $n = 4$.



This type of adder is known as the ripple carry adder (RCA) as the carry propagates from one stage to the next inside the circuit.



The fact that this design is a cascade of identical full adder circuits is a good thing as such regularity greatly simplifies the construction of an n -bit adder and allows for an optimal implementation in terms of silicon area and power consumption. The only real issue with this structure, and this is a big one unfortunately, is that it is slow. It indeed takes a long time to obtain the result of an addition because of the many gates that the signal must go through before being able to produce the final sum bit S_5 .

To illustrate this point, let us evaluate the time needed by a four-bit RCA to compute the sum of two numbers A and B .

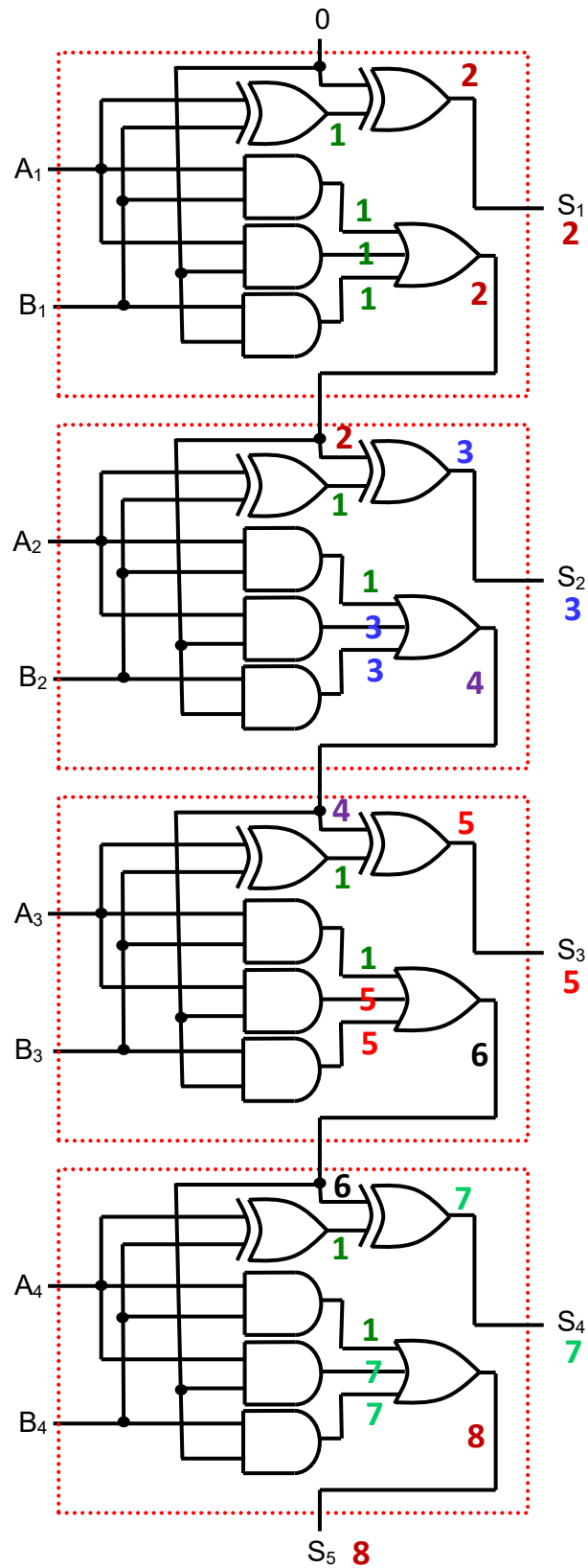
Assume that, at time $t = 0$, both numbers $A = (A_4A_3A_2A_1)$ and $B = (B_4B_3B_2B_1)$ are made available on the circuit inputs. Also assume, for simplicity's sake, that all logic gates introduce the same delay, called propagation time and denoted as t_p , to a signal passing through them.

The diagrams below provide a graphical description of the way the various signals propagate through the circuit as a function of time.

A number associated with a node indicates the multiple of time t_p needed by the signal to settle to its final value at this particular node. The computation delay of the whole circuit is thus the highest time multiple among all output nodes S_1 , S_2 , S_3 , S_4 , and S_5 .

It can be observed that the times required to compute the successive sum bits are

- $t_{ps1} = 2t_p$ for the first sum bit S_1 ;
- $t_{ps2} = 3t_p$ for the second sum bit S_2 ;
- $t_{ps3} = 5t_p$ for the third sum bit S_3 ;
- $t_{ps4} = 7t_p$ for the fourth sum bit S_4 ;
- $t_{ps5} = 8t_p$ for the fifth sum bit S_5 .



As for the carry out bits, we obtain the following computation times:

- $t_{pc1} = 2t_p$ for the first carry out bit $C_{out,1}$;

- $t_{pc2} = 4t_p$ for the second carry out bit $C_{out,2}$;
- $t_{pc3} = 6t_p$ for the third carry out bit $C_{out,3}$;
- $t_{pc4} = 8t_p$ for the fourth carry out bit $C_{out,4}$.

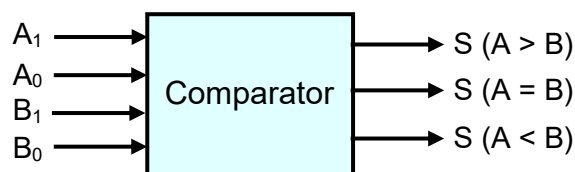
The computation time, $T_{RCA,n=4}$, of a four-bit ripple carry adder is therefore given by $T_{RCA,n=4} = 8t_p$, where t_p is the propagation time through a single logic gate.

As for an addition of two n -bit numbers, one could easily show that the computation time of an n -bit RCA is given by $T_{RCA,n} = 2nt_p$. For instance, the computation time would be $T_{RCA,64} = 128t_p$ for a 64-bit RCA. That is considered to be too slow for most applications.

These results could be refined by taking into account the fact that different types of logic gates often have different propagation times. Also, it is certainly possible to design a full adder so as to minimise computation times. For instance, we can reduce these by replacing the AND-OR structure by a NAND-NAND configuration; NAND gates are indeed significantly faster and simpler to design than their AND and OR counterparts. We can also think about designing at the transistor level rather than at the logic gate level, which is generally a faster alternative.

In any case, the most important point to remember at this stage is that the ripple carry adder (RCA) needs a lot of time, relatively speaking, to generate the result of an addition, especially when dealing with large binary numbers. The critical path for computation is the path taken by the carries to propagate from one stage to the next. In fact, the reason why the RCA is slow is that it lacks parallelism as the computation of the successive carries is done sequentially.

Example 2: Design of a comparator of two-bit numbers A and B, with four inputs and three outputs. The symbol and truth table of this circuit are given below.



A ₁	A ₀	B ₁	B ₀	S (A > B)	S (A = B)	S (A < B)
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

Output S (A > B):

		A ₁ A ₀			
		00	01	11	10
B ₁ B ₀	00		1	1	1
	01		$A_0 \overline{B_1} \overline{B_0}$	1	1
	11				$A_1 \overline{B_1}$
	10			1	$A_1 A_0 \overline{B_0}$

$$S(A > B) = A_1 \overline{B_1} + A_0 \overline{B_1} \overline{B_0} + A_1 A_0 \overline{B_0} = A_1 \overline{B_1} + A_0 \overline{B_0} (\overline{B_1} + A_1)$$

Output S (A = B):

		A ₁ A ₀			
		00	01	11	10
B ₁ B ₀	00	1			
	01		1		
	11			1	
	10				1

$$\begin{aligned}
 S(A = B) &= \overline{A_1} \overline{A_0} \overline{B_1} \overline{B_0} + \overline{A_1} A_0 \overline{B_1} B_0 + A_1 A_0 B_1 B_0 + A_1 \overline{A_0} B_1 \overline{B_0} \\
 S(A = B) &= \overline{A_1} \overline{B_1} (\overline{A_0} \overline{B_0} + A_0 B_0) + A_1 B_1 (A_0 B_0 + \overline{A_0} \overline{B_0}) \\
 S(A = B) &= (\overline{A_1} \overline{B_1} + A_1 B_1) (\overline{A_0} \overline{B_0} + A_0 B_0) = \overline{A_1 \oplus B_1} \cdot \overline{A_0 \oplus B_0}
 \end{aligned}$$

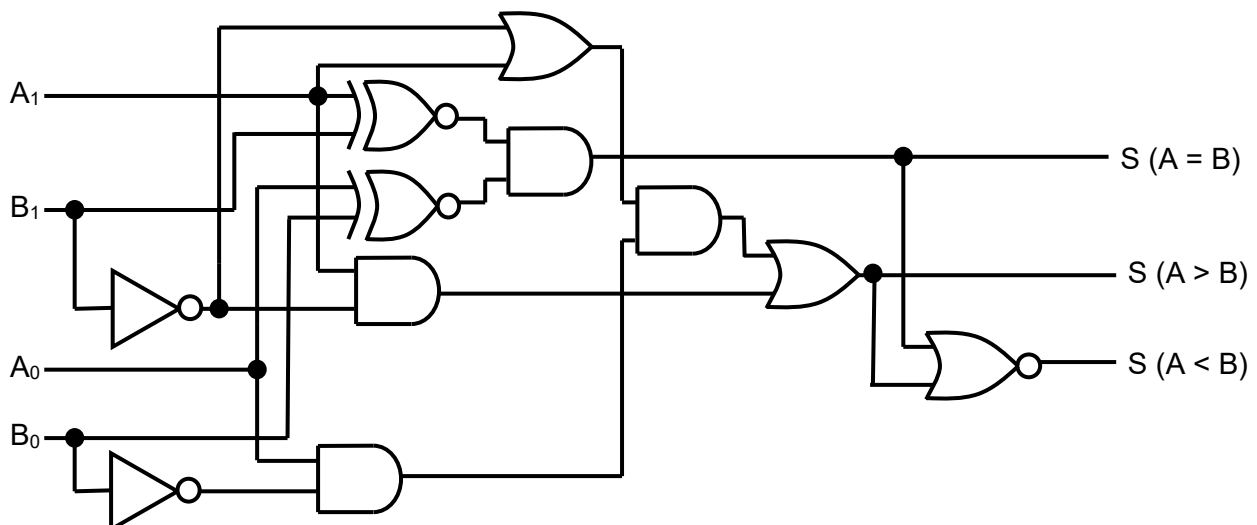
We do not need to repeat the Karnaugh map procedure for the third output $S(A < B)$ since we notice that $S(A < B)$ is equal to 1 if and only if outputs $S(A > B)$ and $S(A = B)$ are both equal to 0. Thus $S(A < B)$ can be obtained using a simple two-input NOR gate, as demonstrated in the truth table below:

$$S(A < B) = \overline{S(A > B) + S(A = B)}.$$

$S(A > B)$	$S(A = B)$	$S(A < B)$
0	0	1
0	1	0
1	0	0
1	1	X

The value X means either 0 or 1 at the designer's discretion, because the combination $S(A > B) = S(A = B) = 1$ cannot exist. We chose to adopt $X = 0$ here and thus ended with a NOR gate. If we chose $X = 1$ instead, we would have ended with an XNOR gate which is slightly more complex to implement than a NOR gate.

The logic diagram thus obtained is depicted below.



Note that this structure is not regular at all, which can be a problem when trying to implement this circuit in an actual silicon chip.

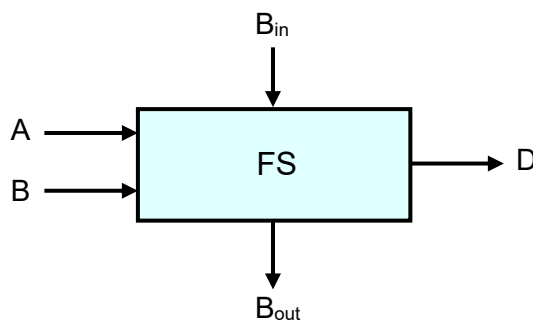
In fact, we would probably not design a comparator in this way in practice. Instead, we would use an n -bit subtractor circuit to perform the comparison between two n -bit numbers (or string of bits) A and B . Then, we would examine the result $D = A - B$ of this subtraction and draw conclusions as follows:

- If the sign of D indicates that D is positive, we would conclude that $A \geq B$;
- If the sign of D indicates that D is negative, we would conclude that $A < B$;
- If $D = 0$, we would conclude that $A = B$.

The “subtraction” method is the way comparisons are done in general-purpose processors and many other circuits that make use of comparators.

Example 3: Design of a full subtractor.

A full subtractor is a circuit that subtracts two bits from another bit. In other words, it performs the operation $D = A - B - B_{in}$ on three bits denoted A , B , and B_{in} . A full subtractor must thus have two binary outputs, namely the difference D as well as a borrow B_{out} . A full subtractor’s symbol and truth table are shown below.



A	B	B _{in}	D	B _{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0

1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

The Karnaugh map for output D is as follows:

		A B			
		00	01	11	10
B_{in}	0		1		1
	1	1		1	

$$D = \overline{A}\overline{B}B_{in} + \overline{A}B\overline{B_{in}} + AB B_{in} + A\overline{B}\overline{B_{in}} = \overline{B_{in}}(\overline{A}B + A\overline{B}) + B_{in}(AB + \overline{A}\overline{B})$$

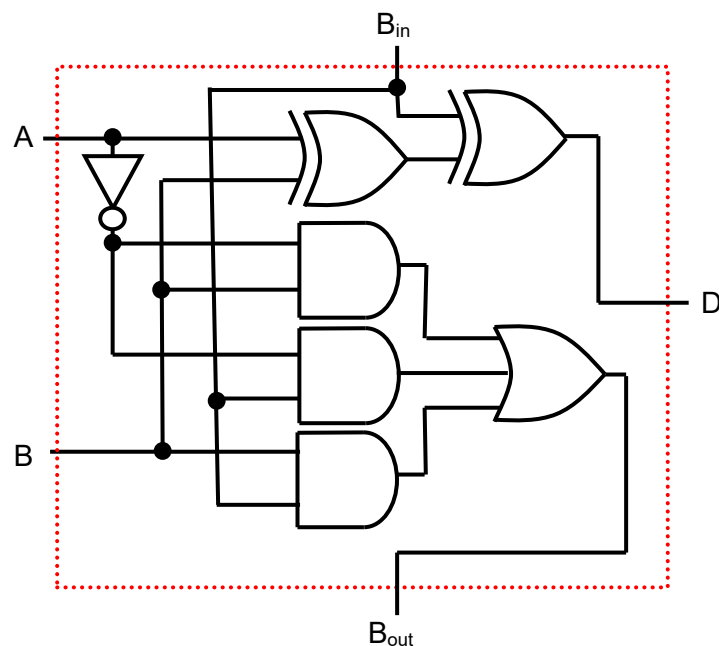
$$D = \overline{B_{in}}(A \oplus B) + B_{in}(\overline{A \oplus B}) = B_{in} \oplus (A \oplus B)$$

The Karnaugh map for output B_{out} is as follows:

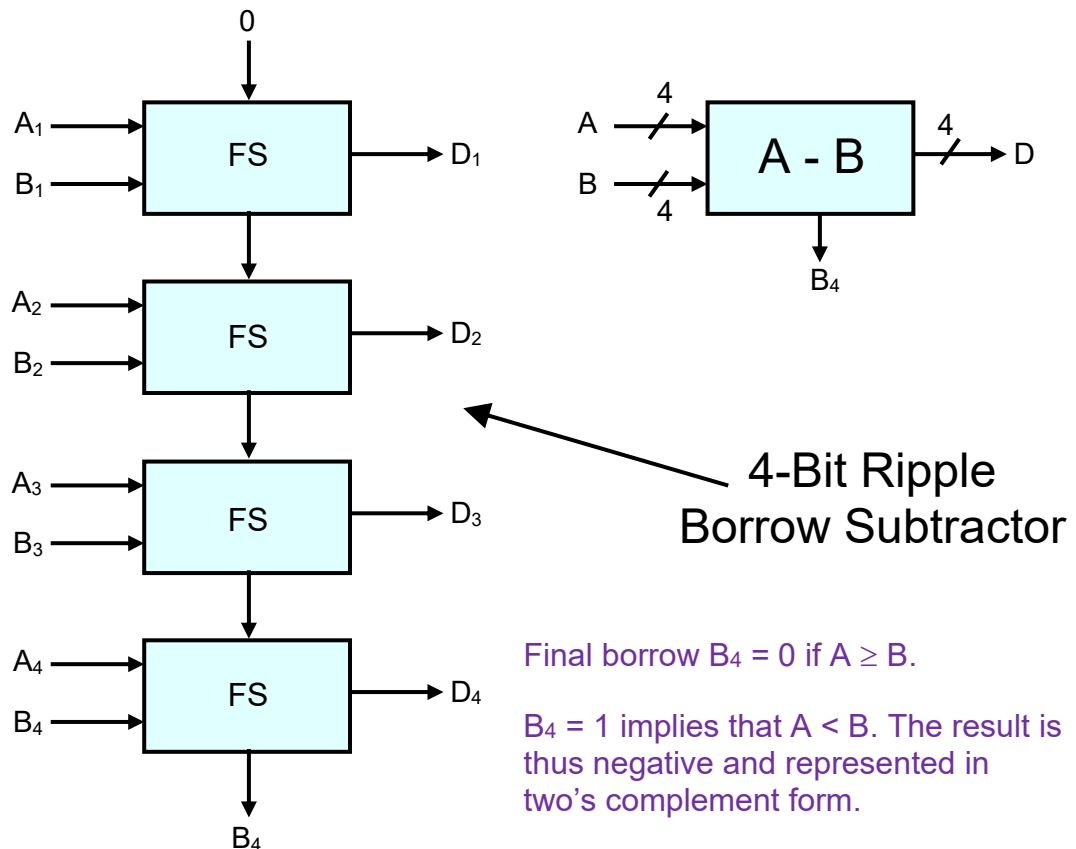
		00	01	11	10
	0		1		
	1	1	1	1	

$$B_{out} = \overline{A}B_{in} + \overline{A}B + BB_{in}$$

The corresponding circuit is displayed below.



A full subtractor can be used as a basic building block for designing an n -bit subtractor, i.e. a combinational logic circuit that performs the subtraction of two n -bit numbers. We give below an example for $n = 4$. This circuit is known as the ripple borrow subtractor.



Example 4: Design of an n -bit adder/subtractor.

We are going hereafter to show that it is possible to design a circuit that can function as either an n -bit adder or an n -bit subtractor. The user can select the mode of operation with a control bit, called AS , as follows:

- If $AS = 0$, the circuit behaves as an n -bit adder;
- If $AS = 1$, the circuit functions as an n -bit subtractor.

The full adder and the full subtractor, previously designed, are two circuits that look very much alike. Hence, one can legitimately wonder whether it would be possible to combine both circuits into a single one that could be simply programmed to perform either an addition or a subtraction. More specifically, we would like to implement a full subtractor using a full

adder. The latter would be slightly modified every time a subtraction, rather than an addition, is required.

We found the two following Boolean expressions for a full-adder:

$$S = C_{in} \oplus (A \oplus B) \text{ and } C_{out} = AB + AC_{in} + BC_{in}.$$

As for the full subtractor, we obtained the Boolean equations

$$D = B_{in} \oplus (A \oplus B) \text{ and } B_{out} = \bar{A}B_{in} + \bar{A}B + BB_{in}.$$

It is clear that the only difference between both sets of equations is the variable $\bar{A}$ replacing the variable A in the expression of B_{out} .

If we chose to apply $\bar{A}$ on the A input of a full adder, the C_{out} output would produce the correct B_{out} value for a subtractor. However, that would not work because, in this case, the same full adder's output S would then be calculated using the expression $S = C_{in} \oplus (\bar{A} \oplus B)$, which is different from the equation $D = B_{in} \oplus (A \oplus B)$ associated with a subtractor. The answer is clearly not straightforward and finding it requires a bit more thinking.

The trick consists of redesigning a full subtractor so that it processes an inverted borrow $\bar{B}_{in}$ and generates an inverted borrow $\bar{B}_{out}$. This does, fundamentally, not change anything to the overall operation of such circuit, but will provide a solution to the problem we are trying to solve.

The new expression needed to compute $\bar{B}_{out}$ is, as usual, obtained using a Karnaugh map.

	A B				
	00	01	11	10	
0	1		1	1	$\bar{B} \bar{B}_{in}$
1				1	

$\bar{A} \bar{B}_{in}$ (green arrow pointing to cell 11, row 1)
 $A \bar{B}$ (red arrow pointing to cell 10, row 1)

$$\bar{B}_{out} = \bar{A} \bar{B}_{in} + \bar{B} \bar{B}_{in} + A \bar{B}$$

So, we have just shown that a full subtractor can also be defined by the following equations:

$$D = B_{in} \oplus (A \oplus B) \text{ and } \overline{B_{out}} = A\overline{B_{in}} + \overline{B} \overline{B_{in}} + A\overline{B}.$$

It is possible to proceed further by using two interesting properties of XOR gates which are as follows:

$$(1) X \oplus \overline{Y} = XY + \overline{X} \overline{Y} = \overline{X \oplus Y},$$

$$(2) \overline{X} \oplus \overline{Y} = X \oplus Y.$$

Therefore, the expression $D = B_{in} \oplus (A \oplus B)$ can be re-written as $D = \overline{\overline{B_{in}} \oplus (A \oplus B)} = \overline{\overline{B_{in}} \oplus (A \oplus \overline{\overline{B}})} = \overline{B_{in}} \oplus (A \oplus \overline{B})$.

The new subtractor's expressions $D = \overline{B_{in}} \oplus (A \oplus \overline{B})$ and $\overline{B_{out}} = A\overline{B_{in}} + \overline{B} \overline{B_{in}} + A\overline{B}$ are strictly identical to those of a full adder provided that $\overline{B}$, $\overline{B_{in}}$, D , and $\overline{B_{out}}$ are replaced by B , C_{in} , S , and C_{out} , respectively.

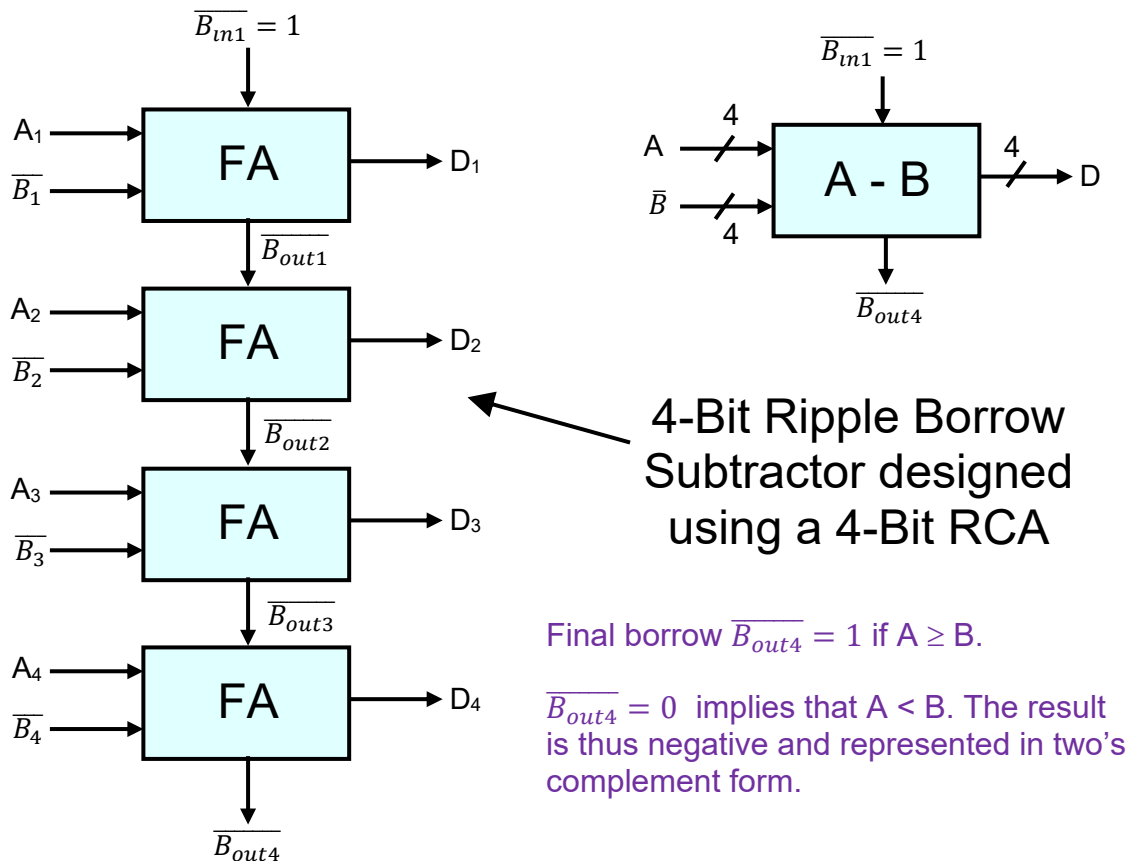
This shows that a subtraction $D = A - B$ can be performed using a full adder circuit if $\overline{B}$ and $\overline{B_{in}}$ are applied to the B and C_{in} inputs. The difference D is then available on the S output, whereas the inverse, $\overline{B_{out}}$, of the output borrow appears on the C_{out} output.

An n -bit subtractor can thus be designed using only full adder circuits. We give below an example for $n = 4$.

Let us now put everything together to design a full adder/subtractor that can perform either an addition or subtraction depending on the value of a bit, called AS , as follows:

- If $AS = 0$, the circuit will behave as a full adder;
- If $AS = 1$, the circuit will behave as a full subtractor.

The core circuit is a full adder. If $AS = 0$, then its B input is fed with B , whereas, if $AS = 1$, then its B input is fed with $\overline{B}$. In other words, the full adder's B input is connected to the output of a logic gate having the truth table shown below.



AS	B	Output
0	0	0
0	1	1
1	0	1
1	1	0

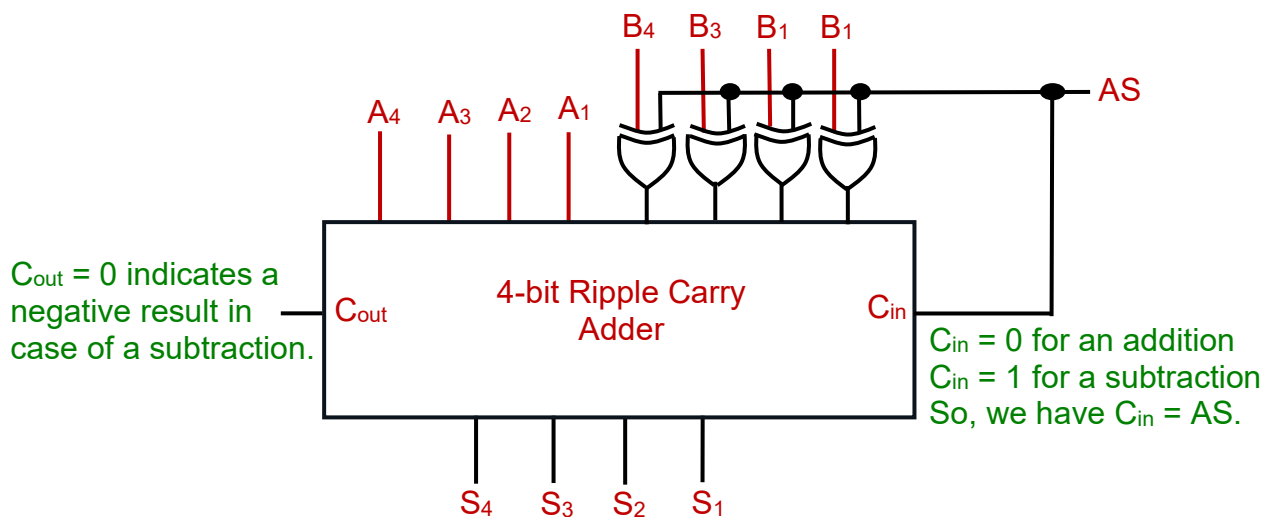
This is the truth table of an XOR gate.

Finally, an n -bit adder/subtractor can be designed using n full adder circuits combined with n XOR gates, as depicted below for the case where $n = 4$.

We remember that inverting all the bits of an n -bit number $B = (B_n \dots B_3 B_2 B_1)$ and then adding the decimal value 1 to it (by setting C_{in} to 1 instead of 0) is equivalent to converting this number from a natural to a two's complement representation. In such case, the n -bit RCA actually performs the addition $A + (2^n - B) = 2^n + (A - B)$. We recognise the

subtraction term, $A - B$, in this result, which is exactly what we wanted. However, there are then two possibilities:

- If $A \geq B$, then the final borrow of the subtraction $A - B$ is zero, which means that the final carry out produced by the n -bit RCA is then equal to one due to the presence of the term 2^n in the final result. Hence, a final carry out equal to one indicates that $A \geq B$. This is consistent with the fact that the final full adder generates $\overline{B_{out,n}}$, that is the inverse of the final borrow $B_{out,n}$.
- If $A < B$, then the final borrow of the subtraction $A - B$ is one. This constitutes a warning to the user that the result is negative and thus erroneous in its present form. In such case, the final carry out produced by the n -bit RCA is then zero because a final borrow equal to one cancels out the term 2^n in the final result. Hence, a final carry out equal to zero indicates that $A < B$, and hence a negative result. Once again, this does not come as a surprise once we remember that the final full adder generates the inverse of the final borrow $B_{out,n}$ on its C_{out} output.



Example 5: Design of a two-bit multiplexer.

A two-bit multiplexer is a digital circuit with three inputs, two data inputs A and B , as well as a control input C , and one output X defined so that $X = A$ if $C = 0$ and $X = B$ if $C = 1$.

A multiplexer can have more than two data inputs. In general, a 2^n -bit multiplexer is a circuit that selects a particular input among a set of 2^n data inputs according to the values of n

control inputs. This circuit implements a fundamental function that is the hardware version of the conditional instruction

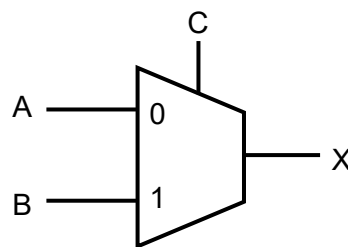
```

if (condition = true) then
    do something
else
    do something else
endif

```

which is so often found at the core of all computer programs.

The symbol and the truth table of a two-bit multiplexer are depicted below.



A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

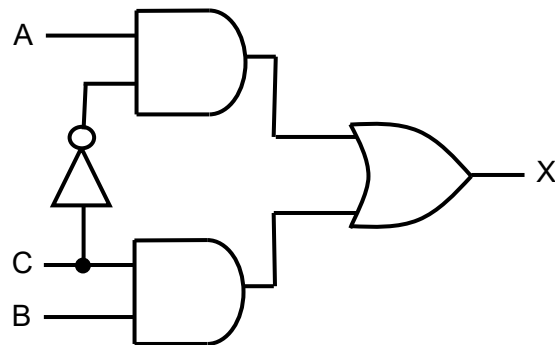
The Karnaugh map for output X is as follows:

	A B			
	00	01	11	10
0			1	1
1		1	1	$A\bar{C}$

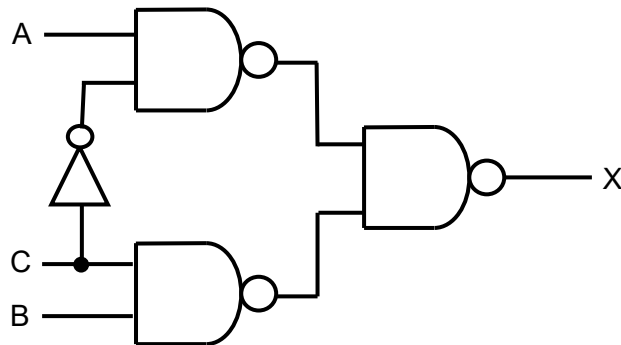
BC

$$X = A\bar{C} + BC$$

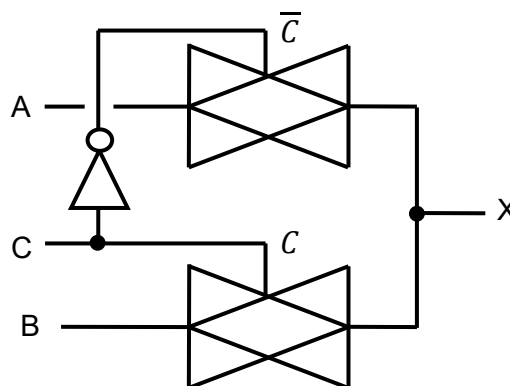
The corresponding circuit is shown below.



Or, in case we prefer using NAND gates (as they are faster and simpler to implement than AND and OR gates):

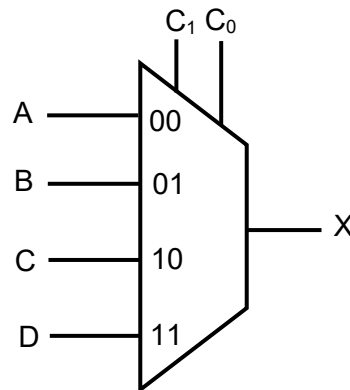


A two-bit multiplexer can also be designed by employing transmission gates:



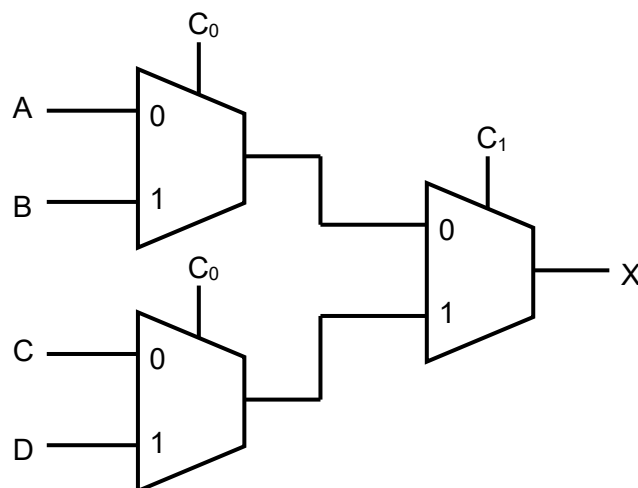
It is straightforward to extend these results to the design of multiplexers with n control inputs and 2^n data inputs. For instance, consider the case $n = 2$ and the corresponding *compact* truth table shown below,

C_1	C_0	X
0	0	A
0	1	B
1	0	C
1	1	D

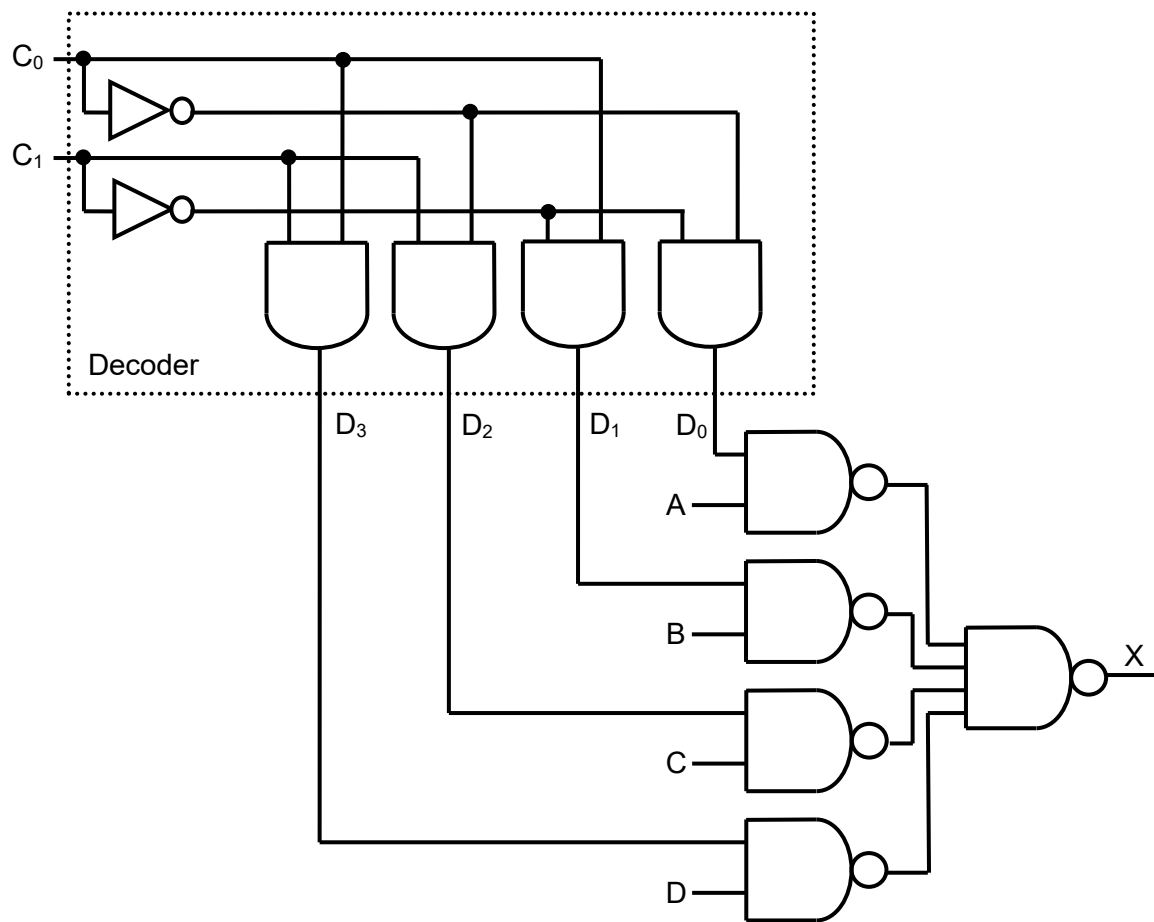


It is easy to see that the multiplexer output is given by $X = A\overline{C_1}\overline{C_0} + B\overline{C_1}C_0 + CC_1\overline{C_0} + DC_1C_0$. Once again, there are various ways to implement this equation, as shown in the three logic diagrams below.

- By cascading two-input multiplexers:



- By relying on a classical NAND-NAND or AND-OR arrangement and using a structure known as a 2-to-4 decoder circuit.



The truth table of a 2-to-4 decoder (2 inputs, 4 outputs) is shown below.

C_1	C_0	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

- By relying on transmission gates and using a 2-to-4 decoder circuit.

